Improved Residual Learning in Diffusion Models

Junyu Zhang, Daochang Liu, Eunbyung Park, Shichao Zhang, *Senior Member, IEEE*, and Chang Xu, *Senior Member, IEEE*

***Abstract*—Diffusion models (DMs) have profoundly transformed the field of generative modeling, delivering exceptional image generation quality. Nevertheless, residuals in the generated images, arising from modeling errors that accumulate along the practical sampling trajectories of pre-trained DMs, remain an inevitable challenge. To mitigate this, we propose a novel residual learning framework built upon a parameterized correction function, designed to improve modeling performance through residual correction. Most notably, our framework exhibits remarkable transferable residual correction capabilities, enabling a correction function optimized for a specific pre-trained DM on a given dataset to enhance the performance of other DMs trained on the same dataset. To further improve our framework, we introduce an advanced training approach that designs an ODE sampler bank to refine residual simulation and incorporates a score consistency maintenance technique to enhance model convergence. Building on this approach, the optimized correction function achieves substantial improvements in residual correction. Extensive experiments performed on four widely used datasets and multiple pre-trained DMs validate the effectiveness and superiority of our residual learning framework.**

***Index Terms*—Diffusion models, residual learning, residual correction, ODE sampler bank, transferable residual correction.**

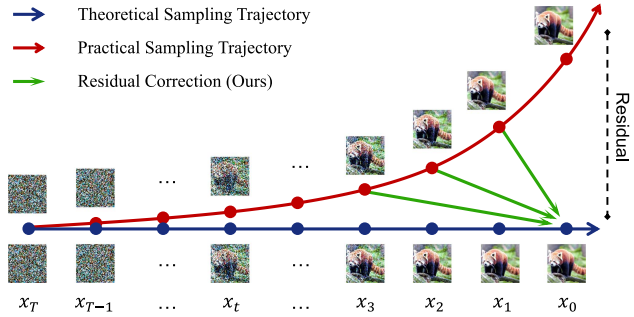


Fig. 1. Residual Learning in Diffusion Models. The modeling error in DMs is an inherent issue, caused by score estimation and discretization errors, which accumulates along the sampling trajectory, leading to a residual in the generated image compared to its theoretical counterpart. This inevitably decreases the image quality. To mitigate this, our residual learning framework employs a parameterized correction function to rectify the practical sampling trajectory through residual correction, thereby improving image quality.

I. Introduction

DIFFUSION models (DMs) [1], [2], [3], a prominent generative paradigm, have long led the artificial intelligence community [4], [5], [6], with applications in areas such as controllable generation [7], [8] and physics-aware simulation [9], [10]. By inheriting their excellent modeling capabilities, DMs have also advanced several practical domains, including 3D reconstruction [11], molecule synthesis [12], and segmentation tasks [13]. Unfortunately, DMs are limited by the modeling error that arises during sampling [14], [15], [16], leading to inevitable residuals in the generated images [17].

To elucidate the residual, we begin by reviewing the diffusion modeling framework, which consists of two processes, *i.e.*, forward diffusion and reverse modeling [18], [19]. In the diffusion process, a forward SDE is employed to perturb the data distribution via a well-designed noise schedule [3], [20]. Since the noise scale gradually reaches its maximum value, the target data distribution reverts to a prior distribution [21]. Reverse modeling can be derived from the forward SDE by considering the score of the marginal probability densities as a function of time [3], [22]. Crucially, this process satisfies a reverse-time SDE or a probability flow (PF) ordinary differential equation (ODE) [3]. We can therefore approximate the reverse-time S/ODE by training a time-dependent deep neural network to estimate the score [1], [23]. Once the score is matched, images can be generated by implementing a discretization strategy, which linearly solves the reverse-time S/ODE with a numerical solver [24], [25], [26].

In light of the physical-inspired diffusion mechanism [18], DMs tend to be based as directly as possible on a rigorous theoretical foundation [20], [22], which provides solid evidence that a modeling gap between the target data distribution and its practical modeling counterpart cannot be avoided [14], [15], [21], [25]. Formally, the resulted practical sampling trajectory gradually deviates from its corresponding theoretical counterparts [14], [16], leading to an increasingly widening gap, as shown in Fig. 1. Consequently, this gap introduces residuals

Received 22 January 2025; revised 5 March 2026; accepted 25 April 2026. Date of publication 4 May 2026; date of current version 6 August 2026. This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) under Grant RS-2024-00457882, in part by AI Research Hub Project under Grant RS-2025-25441838, in part by the Development of a human foundation model for human-centric universal artificial intelligence and training of personnel, and in part by Australian Research Council under Grant DP240101848 and Grant FT230100549. Recommended for acceptance by J. B. Huang. *(Corresponding author: Chang Xu.)*

Junyu Zhang is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea (e-mail: zhangjunyu@skku.edu).

Daochang Liu is with the School of Physics, Mathematics and Computing, University of Western Australia, Crawley, WA 6009, Australia (e-mail: daochang.liu@uwa.edu.au).

Eunbyung Park is with the Department of Artificial Intelligence, Yonsei University, Seoul 03722, South Korea (e-mail: epark@yonsei.ac.kr).

Shichao Zhang is with the Guangxi Key Lab of Multi-Source Information Mining & Security, Guangxi Normal University, Guilin 541004, China (e-mail: zhangsc@gxnu.edu.cn).

Chang Xu is with the School of Computer Science, University of Sydney, Darlington, NSW 2008, Australia (e-mail: c.xu@sydney.edu.au).

Digital Object Identifier 10.1109/TPAMI.2026.3689939

in the generated images when compared to their ground-truth counterparts, which reduces image quality.

The primary reason is that an inevitable issue, known as modeling error [20], occurs in practical modeling, caused by score matching [15], [27] and the discretized sampling strategy [21], [25]. Specifically, for the former, during training, we need to estimate the expected value of the score at each noise scale [3]. However, in practice, obtaining the exact score is intractable as it requires access to all samples. To remedy this, the solution is approximated using the law of large numbers [1], [2], which inevitably introduces the score estimation error [15]. As for the latter, the error source is in that we cannot directly solve the approximated reverse-time S/ODE due to the high dimensionality of the data [23]. Instead, the reverse-time S/ODE needs to be discretized as a sampling trajectory that consists of finite time steps [28], [29]. An approximate solution can be obtained by using numerical solvers [25], [26] to iteratively solve the integrals [6], [20]. However, the discrete sampling strategy introduces discretization error, attributed to its linear nature when solving the integral [30], [31]. Since this involves the matched score, these two errors contribute simultaneously to the modeling error, which accumulates along the practical sampling trajectory [15], [32]. Hence, the generated images exhibit residuals due to the accumulated modeling errors.

Inspired by the observations above, we put forward a novel residual learning framework to mitigate the residuals. The core idea is to employ a parameterized correction function to rectify practical sampling trajectories through residual correction, as shown in Fig. 1. Conceptually, we disassemble the sampling process into two phases. In the first phase, a diffusion sampler [33], [34] generates sampling trajectories starting from the prior distribution and ending at an intermediate time step, which approaches the final sampling step. In the subsequent phase, since the trajectories deviate from their theoretical counterparts [35], our correction function guides them to converge toward the target data distribution. Building on this correction mechanism, our framework delivers significant performance in residual reduction, eliminating the need for fine-tuning or training DMs from scratch.

We further design an advanced training approach for the correction function by incorporating an ODE sampler bank and the score consistency technique. For the former, the bank consists of renowned PF ODE diffusion samplers, designed to simulate varying magnitudes of residuals. Concretely, the practical trajectories randomly traverse the sampling path, causing the residuals to fluctuate within a value range, especially for trajectories generated by different DMs. Since directly obtaining the value range is intractable, we instead use different samplers to simulate it by constructing the trajectories with the same matched score. Accordingly, the residuals show slight variations due to the different formulations of the samplers, effectively simulating the residual patterns observed in real-world scenarios. For the latter, inspired by the score distillation loss [36], we maintain score consistency during training to boost the convergence of the correction function. Theoretically, under the same noise-level perturbation, the score of the generated images after residual correction should align with that of their ground-truth counterparts. Hence, minimizing their practical discrepancy will naturally benefit convergence. Following this advanced training philosophy, the converged correction function enhances the ability to perform both transferable and effective residual corrections.

The resulting correction function, trained for a specific DM, can improve arbitrary sampling trajectories constructed by different DMs trained on the same dataset, demonstrating impressive transferable residual correction capability. This is because the diffusion modeling paradigm essentially constructs a sampling path between the prior distribution and the target data distribution [29], [37]. In this context, for a given dataset, different DMs aim to estimate the same ground-truth score. Consequently, the score matched by different DMs becomes the only variable during sampling, with only slight variations arising from their model capacities. As a result, the sampling trajectories generated by different DMs trained on the same dataset exhibit similar trends, leading to a common residual pattern. The correction function optimized for a given pre-trained DM captures this pattern, thereby enabling the enhancement of various DMs trained on the same dataset in a plug-and-play fashion. Leveraging this, for a given dataset, we need only train a single correction function based on a specific pre-trained DM, which can then facilitate various DMs trained on the same dataset in a plug-and-play manner.

Our previous work [38], presented at the conference, primarily focuses on effective residual correction. Compared with the previous conference version, this paper makes significant extensions to the framework in two key aspects. First, we introduce a novel ODE sampler bank to simulate varying magnitudes of residuals by utilizing multiple renowned diffusion samplers. This allows the framework to effectively capture and address residual patterns encountered in real-world sampling scenarios, enhancing the robustness of residual correction. Second, we propose a score consistency technique inspired by score distillation loss, which aligns the corrected scores of generated images with their ground-truth counterparts under similar noise-level perturbations. This addition facilitates faster convergence and improves the generalization of the correction function. These enhancements are validated through extensive experiments, demonstrating notable improvements in both effective and transferable residual corrections.

II. Related Works

Diffusion Models: The diffusion framework [1], [2], [3], [39], [40] represents a family of generative paradigms that facilitates image synthesis by leveraging principles of non-equilibrium thermodynamics [20], [22]. With this theoretical guarantee, when compared to previous generative models, including VAEs [41], flow-based models [37], and GANs [42], DMs are capable of achieving remarkable performance [43], [44]. To further boost DMs, several remarkable techniques have been introduced, including diffusion in latent space [5], [6], the design of dynamic time-dependent weighting schedule [15], [27], [40], and the application of fundamental physical equations [9], [45]. Moreover, DMs exhibit advanced capabilities

in controllable generation [46], [47], encompassing text-to-image [8], [48], text-to-video [49], [50], and text-to-3D rendering [51], [52], all of which hold significant practical value [53]. On the other hand, DMs also demonstrate unparalleled advancements in various domains [54], [55], [56], particularly in the field of AI for scientific research [57]. Despite their impressive performance, DMs are prone to unavoidable modeling errors that accumulate along the sampling trajectories [35], resulting in residuals in the generated images.

Modeling Error: The modeling error is intrinsic to the nature of diffusion modeling [58], [59], [60], [61]. To address this, three main approaches have been proposed, namely training-free numerical solvers [24], diffusion distillation [62], and consistency trajectory training [28]. The first line of work focuses on designing sophisticated diffusion samplers by utilizing high-order numerical solvers to mitigate modeling errors [25], [26], especially under the requirement for fast sampling. Recently, the diffusion distillation technique [30], [63] has greatly reduced the modeling error in fewer sampling steps by distilling knowledge from larger sampling steps. Unlike the approaches mentioned above, consistency models [28], [29] straighten the sampling trajectories to reduce the complexity of integral solving, thereby minimizing the modeling error. Moreover, several interesting works have also been proposed to achieve the same goal. For instance, DMCMC [21] initializes the sampling trajectory near the target data distribution, thus reducing error accumulation. Additionally, [14], [32] propose adjusting the matched score using a robust discriminator to decrease the score estimation error. More recently, DGSolver [61] was proposed to address this issue in restoration tasks via universal posterior sampling. Orthogonal to all the above, we propose a residual learning framework that employs an optimized correction function to mitigate residuals.

Transfer Learning: Transfer Learning has become a foundational paradigm in machine learning [64], [65], enabling the adaptation of models trained on one domain to tasks in different but related domains [66], [67]. In generative modeling, recent works propose transferring the weights of pre-trained models from a source domain (e.g., faces) to a new domain (e.g., portraits of one person) [68], [69], [70]. In this paper, the learned transferable correction capability is based on a specific pre-trained DM, and we can still use this capability to rectify the sampling trajectories constructed by various DMs trained on the same dataset. Moreover, domain adaptation techniques, such as adversarial domain alignment [71], [72] and domain-specific normalization [73], have further advanced transfer learning by mitigating domain shifts between source and target domains [74]. Analogously, we design an ODE sampler bank to further enhance the transferable correction capability, which learns the shifts via simulating different magnitudes of residuals in practical sampling scenarios.

III. Preliminary

A. Elucidating Modeling Error

In this section, we analytically elucidate the phenomenon of modeling error accumulation. Theoretically, the diffusion modeling framework [1], [2] first builds the diffusion path between the target data distribution and the prior distribution. The image generation process is subsequently accomplished by simulating the diffusion path in the reverse direction.

To be specific, for a given target dataset, we assume that its D-dimensional images follow a probability distribution $x_0 \sim p_{\text{data}}(x_0)$. The diffusion path between $p_{\text{data}}(x_0)$ and the prior distribution π can be defined by the following forward SDE:

$$dx = \boldsymbol{F}_t x dt + \boldsymbol{G}_t d\omega, \tag{1}$$

where $\boldsymbol{F}_t \in \mathbb{R}^{D\times D}$ and $\boldsymbol{G}_t \in \mathbb{R}^{D\times D}$ represent the linear drift coefficient and the diffusion coefficient, respectively. Moreover, ω denotes the Brownian motion and $t \sim U[0,1]$. Under some mild assumptions [3], the forward SDE in (1) is associated with a reverse-time diffusion process [34]:

$$dx = \left[\boldsymbol{F}_t x - \frac{1+\lambda^2}{2}\boldsymbol{G}_t\boldsymbol{G}_t^T \nabla \log p_t(x)\right] dt + \lambda \boldsymbol{G}_t d\bar{\omega}, \tag{2}$$

where $\bar{\omega}$ denotes a standard Wiener process in the reverse-time direction, and $\nabla \log p_t(x)$ refers to the gradient of the log probability density with respect to the perturbed data at time step t, a.k.a. score [23], [75]. Here, we introduce $\lambda \geq 0$ to represent a family of SDEs, where $\lambda = 0$ corresponds to the PF ODE [25]. Since $\boldsymbol{G}_t$ and $\boldsymbol{F}_t$ are determinated by the forward SDE, a new sampling trajectory can be constructed by first initializing a sample from the prior distribution π, and the complete trajectory can be obtained by solving (2) [3].

Discretization Error: However, directly solving the integral in (2) is intractable. Alternatively, in practice, the complete integral over $t \sim U[0,1]$ is approximated by dividing it into $N+1$ discretization steps using N sub-intervals, $[t_0 = \epsilon, \ldots, t_s, \ldots, t_N = 1]$. The sum of the integrals over each interval, from time step t_{s+1} to t_s, is equal to the integral over the entire range, from time step t_N to t_0. For simplicity, we adopt the PF ODE by setting $\lambda = 0$ in (2) to elucidate this process. In this context, the integral between time steps t_{s+1} to t_s can be expressed as:

$$\begin{aligned} x_{t_s} = {} & e^{\int_{t_{s+1}}^{t_s} \boldsymbol{F}_\tau d\tau} x_{t_{s+1}} \\ & + \frac{1}{2}\int_{t_{s+1}}^{t_s} \left[e^{\int_{t_{s+1}}^{t_s} \boldsymbol{F}_\tau d\tau} \nabla \log p_\tau(x)\right] d\tau. \end{aligned} \tag{3}$$

Clearly, solving the integral term in (3) remains intractable. In practice, one can utilize a first-order numerical solver [20], [25], [26] to tackle this issue:

$$\begin{aligned} \tilde{x}_{t_s} = {} & e^{\int_{t_{s+1}}^{t_s} \boldsymbol{F}_\tau d\tau} x_{t_{s+1}} \\ & + \frac{t_{s+1}-t_s}{2}\boldsymbol{G}_{t_{s+1}}\boldsymbol{G}_{t_{s+1}}^T \nabla \log p(x_{t_{s+1}}). \end{aligned} \tag{4}$$

As a result, using (4) to approximate the integral defined by (3) introduces a discretization error, which corresponds to the discrepancy between x_{t_s} and $\tilde{x}_{t_s}$.

Score Estimation Error: On the other hand, $\nabla \log p(x_{t_{s+1}})$ in (4) is inaccessible due to the high dimensionality of $x_{t_{s+1}}$, making the probability density function analytically intractable [23]. To remedy this, prior works [1], [75] propose the advanced score matching technique to estimate the ground-truth score $\nabla \log p(x_{t_{s+1}})$, wherein a time-dependent neural network

$s_\theta(x_{t_{s+1}}, t_{s+1})$ is designed to match it:

$$\mathcal{J}_{\mathrm{SM}}(\theta;\omega) = \frac{1}{2}\int_0^1 \mathbb{E}\left[\omega(t_{s+1})\left\|\nabla \log p(x_{t_{s+1}}) - s_\theta(x_{t_{s+1}}, t_{s+1})\right\|_2^2\right] dt.$$

Here, $\log p(x_{t_{s+1}})$ has a closed form expression, which can be derived from a simple Gaussian distribution $p(x_{t_{s+1}}|x_0)$ defined by the given forward SDE [3], by using a pre-defined noise schedule to perturb x_0 [2]. Additionally, $\omega(t)$ denotes a time-dependent weighting function used for stable training, which significantly improves the convergence speed [40].

In practice, $\mathcal{J}_{\mathrm{SM}}(\theta;\omega)$ is optimized using empirical samples via Monte Carlo methods [1], [19], [23]. Since the exact solution of $\nabla \log p(x_{t_{s+1}})$ requires samples from the corresponding distribution $p(x_{t_{s+1}}|x_0)$, this leads to a score estimation error in practice, as we cannot access all of these samples. In addition, previous training approaches [15], [16], [20], [28] involve a trade-off between model likelihood [16] and image quality [5], [14], achieved by setting the smallest time step ϵ close to the target data distribution:

$$\frac{1}{2}\int_\epsilon^1 \mathbb{E}\left[\omega(t_{s+1})\left\|\nabla \log p(x_{t_{s+1}}) - s_\theta(x_{t_{s+1}}, t_{s+1})\right\|_2^2\right] dt. \tag{5}$$

During training, the score at time steps below ϵ cannot be estimated, further aggravating the score estimation error when using the unmatched score for sampling.

Modeling Error: In practice, once the score network $s_{\theta^*}(x_{t_{s+1}}, t_{s+1}) \approx \nabla \log p(x_{t_{s+1}})$ is matched for almost all $x \in \mathbb{R}^D$ and $t_{s+1} \sim U[\epsilon, 1]$, it can be used to generate new images by solving (4) with $\nabla \log p(x_{t_{s+1}})$ replaced by $s_{\theta^*}(x_{t_{s+1}}, t_{s+1})$:

$$\tilde{x}_{t_s} = e^{\int_{t_{s+1}}^{t_s} \boldsymbol{F}_\tau d\tau} x_{t_{s+1}} + \frac{t_{s+1}-t_s}{2} \boldsymbol{G}_{t_{s+1}} \boldsymbol{G}_{t_{s+1}}^T s_{\theta^*}(x_{t_{s+1}}, t_{s+1}). \tag{6}$$

Compared to (4), $\tilde{x}_{t_s}$ introduces a score estimation error arising from the discrepancy between $s_{\theta^*}(x_{t_{s+1}}, t_{s+1})$ and $\nabla \log p(x_{t_{s+1}})$, thereby contributing to the modeling error in addition to the previously derived discretization error. Importantly, modeling errors arise during each integral computation, accumulating throughout the practical sampling trajectories.

B. Residual Definition

Building upon the above investigation, the actual modeling error is the discrepancy between the practical image $\tilde{x}_{t_s}$ and its theoretical counterpart x_{t_s} at time step t_s, which can be expressed as:

$$\Omega(\tilde{x}_{t_s}, x_{t_s}) = x_{t_s} - \tilde{x}_{t_s}.$$

In practice, this error gradually accumulates along the sampling trajectory [3], [28], starting from t_N and propagating to the current step t_s. Hence, the modeling error at time step t_s is accumulated from t_N to t_s, and $\Omega(\tilde{x}_{t_{s+1}}, x_{t_{s+1}})$ represents the recursive error term carried over from the previous step [35], rather than the error arising solely at the current step. Accordingly, we can write the residual at $\tilde{x}_{t_0}$ as:

$$x_0 = \tilde{x}_{t_0} + \Omega(\tilde{x}_{t_0}, x_0). \tag{7}$$

To be explicit, the residual is the discrepancy between the practical image $\tilde{x}_{t_0}$ generated by a DM and its theoretical counterpart x_0, which can degrade image quality. Conversely, compensating $\tilde{x}_{t_0}$ with a learned residual $\Omega(\tilde{x}_{t_0}, x_0)$ naturally yields a better approximation to the ground-truth image x_0.

IV. Residual Learning Framework

To address the residuals that arise in generated images, we put forward a novel residual learning framework that employs a parameterized correction function to rectify practical sampling trajectories, as illustrated in Fig. 2. Building on this foundation, the optimized correction function effectively facilitates residual correction, thereby enhancing overall image quality. Most notably, the correction function, optimized for a specific pre-trained DM on a given dataset, demonstrates transferable residual correction capabilities, enabling it to enhance various DMs trained on the same dataset in a plug-and-play manner.

Below, we introduce residual learning and correction in our framework, elaborate on the optimization process, and provide an in-depth discussion on the transferable residual correction.

A. Overview

In practice, it is challenging to directly quantify the residual $\Omega(\tilde{x}_{t_0}, x_0)$ present in the generated image $\tilde{x}_{t_0}$. This is attributed to the inability to access the corresponding ground-truth image x_0, and therefore, precluding the direct collection of the residual pair $(x_0, \tilde{x}_{t_0})$. Heuristically, we can first acquire x_{t_s} by perturbing the training image x_0 with the corresponding noise scale, and then reverse it to $\tilde{x}_{t_0}$ using an ODE numerical solver. Since the sampling trajectories constructed by an ODE solver are deterministic [24], x_0 and $\tilde{x}_{t_0}$ inherently share similar semantic information. Under such a perspective, we can construct the residual pair $(x_0, \tilde{x}_{t_0})$, thereby enabling a simulation of the residual. Thereupon, we design a two-stage residual learning framework to effectively learn the residual, as conceptually outlined in Algorithm 1.

Residual Learning: In the first stage, we focus on simulating the residuals in the generated images using a deterministic diffusion sampler. Concretely, we utilize a given forward SDE to perturb the training image x_0, and the obtained x_{t_s} resides on the ground-truth sampling trajectory. For simplicity, we employ VP SDE [3] to describe our framework:

$$x_{t_s} = \alpha(t_s)x_0 + \sigma(t_s)z_{t_s}, \tag{8}$$

where $\alpha(t_s)$ and $\sigma(t_s)$ both derived from the pre-designed noise schedule [2], [3], [20], and z_{t_s} denotes the diffusion term sampled from a Gaussian distribution. This is essentially a discrete representation of (1) achieved by setting:

$$\boldsymbol{F}_{t_s} = \frac{\mathrm{d}\log\alpha(t_s)}{\mathrm{d}t_s}, \boldsymbol{G}_{t_s} = \sqrt{\frac{\mathrm{d}\sigma^2(t_s)}{\mathrm{d}t_s} - 2\frac{\mathrm{d}\log\alpha(t_s)}{\mathrm{d}t_s}\sigma^2(t_s)}.$$

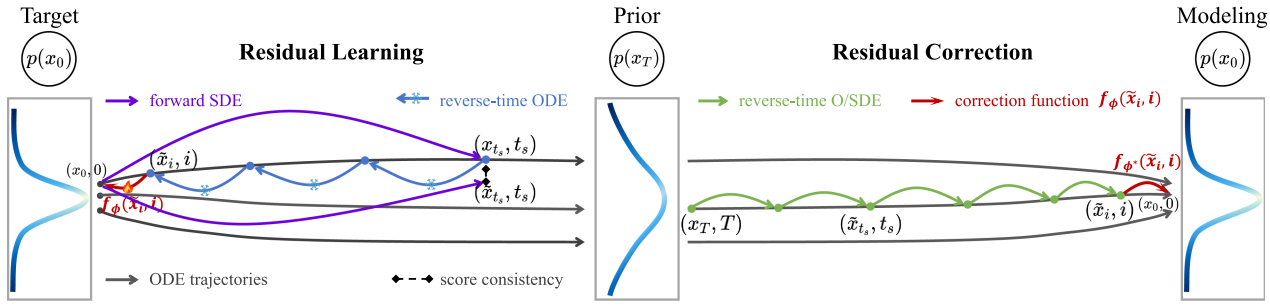


Fig. 2. Overview of **Residual Learning Framework**. For residual learning, in the first phase, we first diffuse x_0 to x_{t_s} using a forward SDE and then randomly select a sampler from the ODE sampler bank to reverse x_{t_s} back to $\tilde{x}_i$. For simplicity, we present only one sampler to illustrate the construction of sampling trajectories. In the next stage, we optimize the parameterized correction function $\boldsymbol{f}_\phi(\cdot,\cdot)$ via the simulated varying magnitude of residuals. Moreover, to maintain score consistency, we minimize the discrepancy between $s_\theta(\tilde{x}_{t_s}, t_s)$ and $s_\theta(x_{t_s}, t_s)$, where $\tilde{x}_{t_s}$ is obtained by replacing x_0 with $\boldsymbol{f}_\phi(\tilde{x}_i, i)$ in (8). For residual correction, a diffusion sampler iteratively diffuse x_T to $\tilde{x}_i$, and an optimized correction function guides $\tilde{x}_i$ toward x_0 accordingly.

Based on the preceding analysis, this forward diffusion process corresponds to a reverse sampling formulation. In this context, the perturbed x_{t_s} can be reversed to $\tilde{x}_{t_0}$ through the iterative solution of (6). As a result, we can effectively simulate the residual by calculating the discrepancy between x_0 and $\tilde{x}_{t_0}$.

In the second stage, to learn the residual, we define the correction function as:

$$\boldsymbol{f} : (\tilde{x}_i, i) \mapsto x_0, \tag{9}$$

where $i \in (\epsilon, t_s)$, and t_s is defined in (8) as the time step selected in the first stage. To avoid unmatched score in $[0, \epsilon]$ mentioned in (5), we utilize $\tilde{x}_i$ instead of $\tilde{x}_{t_0}$ to construct the residual pair. Moreover, it is also difficult to exactly reverse to $\tilde{x}_{t_0}$ due to numerical issues in ODE solving. Therefore, we utilize $\tilde{x}_i$ to compose the residual pair. To define i, we first discretize the continuous time horizon $[\epsilon, 1]$ into N sub-intervals, with boundaries $t_0 = \epsilon < t_1 < \cdots < t_{s-1} < t_s < \cdots < t_N = 1$, where i belongs to $[t_1, \ldots, t_{s-1}]$. It is worth noting that, in practice, ODE samplers can only reverse samples within a limited range back to x_0 due to accumulated modeling errors [76], [77]. Although x_{t_s} cannot be fully denoised to x_0, the resulting $\tilde{x}_{t_0}$ still preserves highly similar semantic information to x_0, and we refer to their discrepancy as the residual. In this context, we define the maximum value of this valid range as t_S, obeyond t_S, the ODE sampler can no longer produce a semantically meaningful output $\tilde{x}_{t_0}$. Therefore, the diffusion time values used in (8) lie within the range $\epsilon < t_s \leq t_S$ and $\epsilon \leq i < t_s$.

Residual Correction: For an optimized correction function $\boldsymbol{f}_{\phi^*}(\cdot,\cdot)$ trained on a given dataset, we can combine it with the practical sampling trajectories approximated by $s_{\theta^*}(x_t, t)$ of arbitrary DM trained on the same dataset. It is worth mentioning that, although $\boldsymbol{f}_{\phi^*}(\cdot,\cdot)$ is optimized using the residuals simulated by the ODE sampler, it can enhance not only deterministic samplers but also stochastic samplers. To be specific, we first randomly sample a batch of samples x_T from the prior distribution and use a diffusion sampler to reverse them to $\tilde{x}_{i^*}$. Subsequently, we utilize the optimized correction function to guide $\tilde{x}_{i^*}$ toward $p_{\text{data}}(x_0)$, where i^* represents the time step at which $\boldsymbol{f}_{\phi^*}(\cdot,\cdot)$ is applied, a hyperparameter discussed in the following paragraph. In this context, the correction function effectively mitigates the residuals in the generated images, as detailed in Algorithm 2.

For the option of the hyper-parameter i^*, we design an adaptive strategy to maximize the effectiveness of our framework. As mentioned earlier, scores near a small ϵ are rarely estimated, leading to imprecise score matching [15]. Owing to the properties of our framework, a large i^* will not influence the performance of our correction function. Hence, we enable to set a relatively large i^* to bypass the unmatched score. On the other hand, ϵ defined in (4) varies greatly, for instance, one can choose $\epsilon = 10^{-2}$ or $\epsilon = 10^{-5}$. Therefore, we establish a unified standard for stable correction with ϵ as reference. Concretely, we set i^* as the nearest and larger step to c and keep this setting across all tasks and datasets, where $c = \max\{0.01, \epsilon\}$. In this manner, our framework can rectify the practical sampling trajectories, leading to a reduction in the residuals and, thereby, improved image quality.

B. Optimization

To unlock the residual correction capability, it is essential to optimize the parameterized correction function. Conceptually, for a given correction function $\boldsymbol{f}(\cdot,\cdot)$, the ultimate goal is enable $\boldsymbol{f}(\tilde{x}_i, i) = x_0$, where $i \in (\epsilon, t]$. Therefore, the training objective can be framed as minimizing the discrepancy between $\boldsymbol{f}(\tilde{x}_i, i)$ and x_0. Suppose we have a free-form deep neural network to parameterize our correction function, we can optimize it by minimizing the following correction loss:

$$\mathcal{L}(\phi; \lambda) = \mathbb{E}_{x_0, \tilde{x}_i}\left[\lambda(i) d(\boldsymbol{f}_\phi(\tilde{x}_i, i), x_0)\right],$$

where $\lambda(\cdot)$ refers to a positive time-dependent weighting function. Additionally, $d(\cdot,\cdot)$ denotes a metric function [78] that satisfies $\forall \mathrm{x}, \mathrm{y} : d(\mathrm{x}, \mathrm{y}) \geq 0$ and $d(\mathrm{x}, \mathrm{y}) = 0$ if and only if $\mathrm{x} = \mathrm{y}$, which is designed to minimize the discrepancy between $\boldsymbol{f}(\tilde{x}_i, i)$ and x_0. In our experiments, we consider the squared l_2 norm $d(\mathrm{x}, \mathrm{y}) = \|\mathrm{x} - \mathrm{y}\|_2^2$, the l_1 norm $d(\mathrm{x}, \mathrm{y}) = \|\mathrm{x} - \mathrm{y}\|_1$, and the Learned Perceptual Image Patch Similarity (LPIPS) [28], [79], all of which are effective for regression tasks. Building on this optimization mechanism, we are able to derive the converged parameters ϕ^* for the correction function.

Algorithm 1: Residual Learning.

```
def residual_learning(Net, sigma, N, x_0,
    T_steps, Sampler_choice):
  """
  Net is the pre-trained DM,
  sigma is the noise schedule,
  x_0 are a batch of training images,
  eta1 and eta2 are learning rate,
  Sampler_choice is the time-dependent sampler
    selection strategy (detailed in B part of
    Section 3),
  T_steps denotes the ODE reverse schedule:
  e.g. [n_2, ... n_t-1],
  ODE_sampler_bank = [DDIM, DPM-Solver, DEIS].
  """

  #add noise via VP SDE
  t = randint(epsilon, N)
  z_t = random.normal(0,I)
  x_t = alpha(t) * x_0 + sigma(t) * z_t

  #the last step of ODE reverse
  i = random.choice(T_steps)

  #select a sampler from the ODE sampler bank
  ODE_solver = Sampler_choice(ODE_sampler_bank)

  #use PF ODE to reverse samples
  for j in range(t, i - 1, -1):
    x_t = ODE_solver(Net, x_t, T_steps[j])

  #the last x_t is x_i
  #residual prediction using the correction
    function
  f_x_i = c_function(x_t)

  #update parameters phi via corection loss
  loss = d(f_x_i, x_0)
  phi = phi - eta1 * partial(loss)/partial(phi)

  #maintaining score consistency
  x_hat_t = alpha(t) * f_x_i + sigma(t) * z_t
  score_diff = d(Net(x_t), Net(x_hat_t))

  #update phi via score consistency loss
  phi = phi - eta2 * partial(score_diff)/partial
    (phi)
```

Algorithm 2: Residual Correction.

```
def residual_correction(net, c_function, epsilon
    , x_t, t_steps):
  """
  net is the pre-trained DM
  c_function is a correction function
  epsilon is the smallest step in Eq.(3)
  x_t sampled from prior distribution
  t_steps is the sampling time schedule
  e.g. [80, ... , 0.58, ... , 0.002]
  """

  #adaptive correction strategy
  c = max(0.01, epsilon)

  for t in t_steps:
    #use reverse O/SDE to generate samples
    x_t = O/SDE_solver(net, x_t, t)

    #use our correction function to correct
    if t <= c:
        x_t = c_function(x_t)
        break

  return x_t
```

C. *Transferability*

Adhering to the design philosophy of our framework, the correction function possesses the property that, when optimized for a target pre-trained DM on a specific dataset, it can enhance various DMs trained on the same dataset in a plug-and-play manner, demonstrating a significantly transferable residual correction capability. Below, we explore this superiority through a intuitive justification.

Specifically, within the diffusion modeling framework, different DMs all aim to model a sampling path between $p_{\text{data}}(x_0)$ and the prior distribution, with the path defined by either a reverse SDE or a PF ODE. The image generation process can be achieved by solving the sampling path using the matched score. On the other hand, for a given target dataset, score networks $s_\theta(x_t, t)$ designed by different DMs are all used to match the same ground-truth score, $\nabla \log p(x_t)$, with the only difference in their sampling process being their model parameters θ. Therefore, once the sampler is provided, the sampling trajectories simulated by different DMs follow similar trends when initialized at the same location, thereby sharing the same residual pattern. In this sense, samples $\tilde{x}_{i^*}$ at the same time step i, modeled by different DMs, fall within a similar domain and can thus be corrected by the correction function optimized on the same dataset. Moreover, the outputs of the correction function are consistent for any pair of $(\tilde{x}_i, i)$ that belong to the same sampling trajectory, i.e., $\boldsymbol{f}(\tilde{x}_a, a) = \boldsymbol{f}(\tilde{x}_b, b)$ for all $a, b \in (\epsilon, t]$. As a result, the optimized correction function can naturally guide samples near $\tilde{x}_{i_{\in(\epsilon,t)}}$ to target data distribution, thus mitigating the residuals. Motivated by this understanding, for a given dataset, we need to optimize only a single correction function, which can boost various DMs trained on the same dataset, offering great efficiency and flexibility.

V. Advanced Training Approach

To further enhance our framework, we propose an advanced training approach that includes the design of an ODE sampler bank and the introduction of a score consistency maintaining technique. Specifically, the ODE sampler bank is designed to refine residual simulation by handling varying magnitudes of residuals, thus improving the model robustness. The latter is employed to facilitate model convergence. By combining them, we significantly improve the optimized correction function, equipping it with both enhanced effective and transferable residual correction capabilities.

A. *Refined Residual Simulation*

In practice, residuals accumulated along the sampling trajectories fluctuate within a specific range, especially in the trajectories generated by different pre-trained DMs. To address this, we design an ODE sampler bank to simulate varying magnitudes of residuals that arise in practical sampling scenarios.

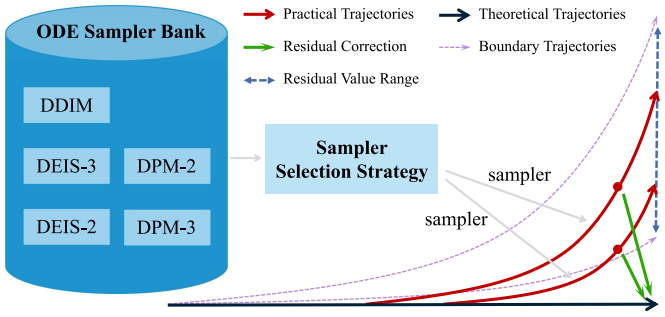


Fig. 3. Simulating Residuals of Varying Magnitudes via a ODE Sampler Bank. For a given dataset, the residuals fluctuate within a certain range, particularly in the images generated by different pre-trained DMs. To effectively simulate this phenomenon, we design an ODE sampler bank and employ a sampler selection strategy to randomly choose a sampler for constructing the current practical trajectory. The trajectories here begin from the intermediate step because we first add noise to x_0 via (8) and then reverse the obtained x_{t_s} using the selected sampler. Since samplers are formulated with different expressions, the generated trajectories exhibit different residuals. When using the correction function to rectify these trajectories, it can learn the shared error patterns, thereby significantly improving the residual correction capability.

ODE Sampler Bank: The ODE sampler bank consisting of DDIM [24], DPM-Slover [26], and DEIS [25], as illustrated in Fig. 3. At a certain time step i, we consider a set of images $\sum_{m=0}^{N}\tilde{x}_i^m$ around $\tilde{x}_i$, which can also be guided toward x_0:

$$\boldsymbol{f}:(\tilde{x}_i^m,i)\mapsto x_0. \tag{10}$$

Accordingly, we employ the ODE sampler bank to simulate slightly different $\tilde{x}_i^*$ compared to $\tilde{x}_i$, with the sampler being randomly selected to reverse x_{t_s} to $\tilde{x}_i$ during each training iteration. More concretely, starting from the same x_{t_s}, different deterministic samplers result in slightly different $\tilde{x}_i$, leading to variations in the magnitude of the residuals. For the DDIM sampler, the detailed formulation according to (8) is:

$$\begin{aligned}\tilde{x}_{t_{s-1}}=&\sqrt{\alpha(t_{s-1})}\frac{x_{t_s}\sigma(t_s)+\sqrt{1-\alpha(t_s)}s_\theta(x_{t_s},t_s)}{\sqrt{\alpha(t_s)}\sigma(t_s)}\\&+\frac{\sqrt{1-\alpha(t_s)}s_\theta(x_{t_s},t_s)}{\sigma(t_s)}.\end{aligned} \tag{11}$$

Here, x_{t_s} is obtained from (1), and then we iteratively apply DDIM until time step i. Similarly, the DPM-Solver with k-order can be expressed as:

$$\begin{aligned}\tilde{x}_{t_s\to t_{s-1}}=&\frac{\alpha(t_{s-1})}{\alpha(t_s)}x_{t_s}-\alpha(t_{s-1})\\&\times\sum_{n=0}^{k-1}\frac{s_\theta^{(n)}(x_{t_s},t_s)}{\sigma(t_s)}\int_{\lambda_{t_s}}^{\lambda_{t_{s-1}}}e^{-\lambda}\frac{(\lambda-\lambda_{t_s})^n}{n!}\mathrm{d}\lambda,\end{aligned} \tag{12}$$

where $\lambda_{t_s}=\log\alpha(t_s/\sigma_{t_s})$, and $s_\theta^{(n)}(x_{t_s},t_s)$ is the n-th order total derivatives of $s_\theta(x_{t_s},t_s)$ w.r.t. λ_{t_s}, both of which can be found in [26]. Importantly, the integral $\int_{\lambda_{t_s}}^{\lambda_{t_{s-1}}}e^{-\lambda}\frac{(\lambda-\lambda_{t_s})^n}{n!}\mathrm{d}\lambda$ can be computed analytically, as detailed in the Appendix of [26]. As for the DEIS sampler, its full expression is:

$$\begin{aligned}\frac{\mathrm{d}x_{t_s}}{\mathrm{d}\rho}&=-s_\theta(\Psi(t_0,t_s)x_{t_s},t_s)/\sigma(t_s),\\\frac{\mathrm{d}\rho}{\mathrm{d}t_s}&=-\frac{1}{2}\sqrt{\frac{\alpha(t_0)}{\alpha(t_s)}}\frac{\mathrm{d}\log\alpha(t_s)}{\mathrm{d}t_s}\frac{1}{\sqrt{1-\alpha(t_s)}},\end{aligned} \tag{13}$$

where ρ is the rescaled time based on t_s, and $\Psi(t_0,t_s)$ is related to $\frac{\mathrm{d}\log\alpha(t_s)}{\mathrm{d}t_s}$, as detailed in [25]. For modeling on a specific dataset, since these samplers simulate the sampling trajectories using the same score network $s_\theta(x_{t_s},t_s)$, they can generate similar $\tilde{x}_i$ with slight residual differences.

Sampler Selection Strategy: To adaptively select a sampler from the ODE bank at each training iteration, we design a strategy that determines which sampler is used to construct the residual. This is crucial because a higher-order sampler requires as many model evaluations as its order to complete a single denoising step. For example, a third-order method such as DPM-Solver-3 must query the pre-trained DM three times to perform one reverse step, while a second-order method such as DEIS-2 requires two queries. Consequently, if the step size $\Delta t=t_s-i$ between the selected time step t_s and the correction step i is misaligned, i.e., not a multiple of the selected sampler's order, a full denoising step cannot be formed. Hence, for a given t_s, it is necessary to determine which sampler can be used at the beginning of each training iteration. Accordingly, we design a strategy based on the step size Δt that adaptively selects a sampler from the corresponding set:

$$\text{sampler}=\begin{cases}\{\text{DDIM},\text{DPM-Solver},\text{DEIS}\}, & \Delta t\mid 6\\ \{\text{DDIM},\text{DPM-Solver-3},\text{DEIS-3}\}, & \Delta t\mid 3\\ \{\text{DDIM},\text{DPM-Solver-2},\text{DEIS-2}\}, & \Delta t\mid 2\\ \{\text{DDIM}\}, & \text{others.}\end{cases}$$

In practice, we have four distinct sets for selecting the sampler based on the time values. Specifically, if the value of Δt is divisible by 3, we select the 3rd-order DEIS, 3rd-order DPM-Solver, or DDIM for constructing the sampling trajectories. If Δt is divisible by 2, we choose the 2nd-order DEIS, 2nd-order DPM-Solver, or DDIM. Moreover, for the special case where Δt mod 6, we randomly select any sampler from the ODE sampler bank. For the other values of time Δt, we can only choose DDIM as the deterministic sampler for constructing the current trajectory. Building on the residual construction mechanism described above, we enable to simulate residuals of varying magnitudes that arise in practical scenarios.

B. Enhanced Optimization Mechanism

To effectively learn the residual, we propose an improved optimization mechanism to further refine the convergence of the parameters, including the data augmentation approach [65] and the score consistency maintenance technique [30], [36].

Data Augmentation: Since data augmentation has long been known to enhance model robustness, employing it is a reasonable choice to strengthen our framework. Motivated by this, we employ several widely used data augmentation methods [80] to reformulate our correction loss:

$$\mathcal{L}(\phi;\lambda,\beta)=\mathbb{E}_{x_0,\tilde{x}_i}\left[\lambda(i)d(\boldsymbol{f}_\phi(\tilde{x}_i,i),x_0)\right]+\mathbb{E}_{x_0,x_0^+}\left[\beta d(\boldsymbol{f}_\phi(x_0^+),x_0)\right],$$

where x_0^+ denotes the corresponding image augmented from x_0, β is a positive weight which performs well across all tasks and datasets when $\beta=1$. In our empirical training, (x_0^+) is obtained using the data augmentation methods we introduced, such as adding small-order Gaussian noise to x_0, rotating and cropping x_0, and masking some pixels [80]. In this manner, the data augmentation operation enables the simulation of implicit samples around $\tilde{x}_i$, thus increasing the diversity of the residuals. To update the parameters of correction function $\boldsymbol{f}_\phi(\cdot,\cdot)$, we minimize $\mathcal{L}(\phi;\lambda,\beta)$ using the stochastic gradient descent algorithm with respect to ϕ. When implementing the learning rate η_1, this process can be formulated as:

$$\phi\leftarrow\phi-\eta_1\bigtriangledown_\phi\mathcal{L}(\phi;\lambda,\beta). \tag{14}$$

Score Consistency: In theory, the output of the optimized correction function should be x_0. In this context, the outputs of the score network should be the same when using $\tilde{x}_{t_s}$ and x_{t_s} to serve as its input, such that $s_\theta(x_{t_s},t_s)=s_\theta(\tilde{x}_{t_s},t_s)$, where $\tilde{x}_{t_s}$ is obtained by applying the same noise level used to perturb x_0 to perturb $\tilde{x}_{t_0}=\boldsymbol{f}_\phi(\tilde{x}_i,i)$. However, in practice, $\tilde{x}_{t_s}$ and x_{t_s} exhibit a discrepancy, causing their score values to differ. Motivated by the score distillation loss [36], to maintain the consistency between $s_\theta(x_{t_s},t_s)$ and $s_\theta(\tilde{x}_{t_s},t_s)$, we introduce the score consistency loss:

$$\mathcal{L}_{\mathrm{SC}}(\phi)=\mathbb{E}_{x_{t_s},\tilde{x}_{t_s}}\left[d\left(s_\theta(\tilde{x}_{t_s},t_s),s_\theta(x_{t_s},t_s)\right)\right],$$

where $\tilde{x}_{t_s}=\alpha(t_s)\boldsymbol{f}_\phi(\tilde{x}_i,i)+\sigma(t_s)z_{t_s}$ via (8), and the metric function used here is the l_2 norm. Analogously, using a learning rate η_2, the parameter update can be expressed as:

$$\phi\leftarrow\phi-\eta_2\bigtriangledown_\phi\mathcal{L}_{\mathrm{SC}}. \tag{15}$$

If the scores $s_\theta(\tilde{x}_{t_s},t_s)$ and $s_\theta(x_{t_s},t_s))$ are equivalent, then $\boldsymbol{f}_\phi(\tilde{x}_i,i)$ is equivalent to x_0, and thus no parameter update is needed. Following the above optimization strategy, during each training iteration, we first update the parameters using (14), and then maintain the score consistency using (15). The conceptually training process can be seen in Algorithm 1.

On the other hand, the exponential moving average (EMA) is used to enhance training stability [20]. That is, given a decay rate $0\le\mu<1$, we perform the following update for ϕ after each optimization step:

$$\phi\leftarrow\mathrm{stopgrad}\left(\mu\phi+(1-\mu)\phi\right),$$

where ϕ is the parameter updated in the last training iteration, and the value of μ follows [20].

VI. Efficient Residual Correction

To avoid exacerbating the issue of slow sampling in DMs, we design an efficient residual correction method, with the correction function parameterized by a lightweight backbone. This is because a lightweight model can definitely reduce the sampling latency. However, optimizing a lightweight correction function using the above training method fails to effectively learn the significant residual capability. Motivated by the recent success of the distillation-based fast sampling framework [63], we leverage this advantage to optimize the lightweight function. Concretely, we distill correction ability from an optimized correction function $\boldsymbol{f}_{\phi^*}(\cdot,\cdot)$ parameterized by heavy parameters ϕ^* to the one with light parameters ϕ^-:

$$\mathcal{L}(\phi^-)=\mathbb{E}_{\tilde{x}_i}\left[d\left(\boldsymbol{f}_{\phi^*}(\tilde{x}_i,i),\boldsymbol{f}_{\phi^-}(\tilde{x}_i,i)\right)\right].$$

Here, the metric function $d(\cdot,\cdot)$ is the squared l_2 norm, defined as $d(\mathrm{x},\mathrm{y})=\|\mathrm{x}-\mathrm{y}\|_2^2$. Similar to the optimize ϕ^*, ϕ^- can also be updated using the stochastic gradient descent algorithm:

$$\phi^-\leftarrow\phi^--\eta_1\bigtriangledown_{\phi^-}\mathcal{L}(\phi^-). \tag{16}$$

Additionally, we utilize the score consistency loss to enhance the convergence of ϕ^-, which can be expressed as:

$$\mathcal{L}_{\mathrm{SC}}(\phi^-)=\mathbb{E}_{\tilde{x}_{t_s}}\left[d(s_\theta(\tilde{x}_{t_s},t_s),s_\theta(\tilde{x}_{t_s},t_s))\right],$$

where $\tilde{x}_{t_s}^-$ and $\tilde{x}_{t_s}$ are derived by perturbing $\boldsymbol{f}_{\phi^-}(\tilde{x}_i,i)$ and $\boldsymbol{f}_{\phi^*}(\tilde{x}_i,i)$ using (8). In this context, we update ϕ^- again:

$$\phi^-\leftarrow\phi^--\eta_2\bigtriangledown_{\phi^-}\mathcal{L}_{\mathrm{SC}}. \tag{17}$$

In each same iteration, we first use (16) to update ϕ^-, and then (17) is applied to update ϕ^- again. Moreover, the EMA strategy is also employed in optimizing ϕ^-, as detailed in the section on the optimization of ϕ^*. Once ϕ^- is optimized, the lightweight correction functions $\boldsymbol{f}_{\phi^-}(\cdot,\cdot)$ are $4\sim64$ times smaller than their heavier counterparts $\boldsymbol{f}_{\phi^*}(\cdot,\cdot)$, with only a slight reduction in image quality.

VII. Experiments

This section introduces the experimental results of the proposed framework. We start by discussing the implementation details, followed by experimental evaluations and ablation studies, and also conclude with a discussion of the comparisons between our previous [38] and current frameworks.

A. Settings and Implementations

Training Setting: Based on our core design philosophy, we use the corresponding model backbone to parameterize the correction function, which is then optimized to enhance the model. To evaluate the transferable correction capability, we use the EDM backbone to parameterize the correction function specifically for CIFAR-10. For transferable correction on ImageNet, since the EDM2 [17] backbones come in multiple versions with varying model sizes, we use all of them to parameterize our correction functions and arbitrarily select one for optimization to enhance the others. During training, we exclude the time embedding setting from the architecture of DM and use it to construct correction function. We retain only the input as a batch of images, and the output is also images, with a slightly reduced model size. Throughout the training process, we maintain consistency in the training settings of the DMs backbone used to construct our correction functions. This includes parameters such as the learning rate, exponential moving average (EMA) decay rate, and noise schedule. We tailor the batch size and training iterations based on the specific characteristics of different correction

Fig. 4. Randomly selected 512 × 512 images improved by our residual learning framework based on DiT [5].

TABLE I
EFFECTIVE RESIDUAL CORRECTION PERFORMANCE ON IMAGENET

Pre-trained Model	FID↓	IS↑
ImageNet 256 × 256 (class-conditional)		
ADM [39]	10.94→[9.55](9.86)	100.98→[114.95](111.01)
ADM-G [39]	4.59→[4.38](4.47)	186.70→[202.86](191.23)
LDM-4 [6]	10.56→[9.97](10.35)	103.49→[121.47](114.62)
LDM-8 [6]	15.51→[13.30](14.47)	79.03→[88.26](84.68)
DiT [5]	9.62→[9.38](9.49)	121.50→[132.19](127.84)
ImageNet 512 × 512 (class-conditional)		
ADM [39]	23.24→[18.76](19.97)	58.06→[66.27](61.54)
ADM-G [39]	7.72→[7.34](7.65)	172.71→[184.61](180.62)
DiT [5]	12.03→[10.73](11.37)	105.25→[128.46](121.53)

Models in the first column represents the baseline pre-trained DMs. The values before → represent the results from the baseline pre-trained DMs. The values after → indicate the results improved by our residual learning framework, where the values in [] are obtained using the correction function optimized based on the training mechanism proposed in this paper, and the values in () are presented in our conference paper [38].

TABLE II
EFFECTIVE RESIDUAL CORRECTION PERFORMANCE ON CIFAR-10

Pre-trained Model	FID↓	IS↑
NSCNv2 [19]	10.87→[9.74](9.89)	8.40→[8.85](8.73)
DDPM [2]	3.17→[2.93](3.05)	9.46→[9.61](9.50)
SDE (VE) [3]	2.55→[2.36](2.44)	9.83→[9.93](9.86)
SDE (deep, VE) [3]	2.20→[2.10](2.15)	9.89→[10.00](9.92)
EDM [20]	2.04→[1.87](1.93)	9.84→[9.99](9.89)

Models in the first column represents the baseline pre-trained DMs. The values before → represent the results from the baseline pre-trained DMs. The values after → indicate the results improved by our residual learning framework, where the values in [] are obtained using the correction function optimized based on the training mechanism proposed in this paper, and the values in () are presented in our conference paper [38].

functions. Specifically, for optimizing the correction functions on CIFAR-10, CelebA, and AFHQv2, we set the batch size to 1024 and the training iterations to range from 40 k to 100 k. In comparison, for ImageNet, we also set the batch size to 1024 and the training iterations to range from 60 k to 100 k.

Empirical Implementation: To evaluate the performance of our residual learning framework, we conduct a series of experiments on four benchmark datasets, including CIFAR-10 [84], CelebA 64 × 64 [85], AFHQv2 64 × 64 [86], ImageNet 256 × 256 and 512 × 512 [87]. Our main experiments consist of two parts, one focuses on verifying the effectiveness of enhancing pre-trained DMs using the correction functions trained for the corresponding DMs, while the other demonstrates the transferable correction capability. Moreover, compared to the performance achieved by the correction function optimized in our conference paper [38], we enhance the correction function using the newly proposed training mechanism. For the first part, we conduct extensive experiments on various datasets with different pre-trained DMs. Specifically, for CIFAR-10, we employ NSCNv2 [19], DDPM [2], SDE VE and VP [3], and EDM [20] to serve as pre-trained DMs. For ImageNet, we utilize ADM [39], LDM [6], and DiT [5] as pre-trained DMs, and the correction function is implemented with the architecture of ADM. For CelebA and AFHQv2, we employ EDM [20], FDM [81], DDPM++ [3], STDDPM [15], Soft Diffusion [82], INDM [83] as pre-trained DMs, and choose the architecture of EDM to formulate the correction function. For the second part, we select CIFAR-10 and ImageNet 512 × 512 as the benchmark datasets. To be specific, for CIFAR-10, we employ a correction function parameterized by an EDM backbone, to enhance various pre-trained DMs. Analogously, for ImageNet, we utilize the correction functions parameterized by EDM2 backbone, to enhance various EDM2 models. This is because EDM2 has multiple versions, each parameterized with different model sizes, as detailed in [17].

B. Experimental Results

In this section, we present evaluations on effective residual correction, transferable residual correction, and efficient residual correction. Moreover, we also compare our method with recent

TABLE III
TRANSFERABLE RESIDUAL CORRECTION ON CIFAR-10

Source Pre-Trained Model: EDM [20]		Correction Functions with Varying Model Sizes				
Target Pre-Trained Model↓	Origin FID↓	C-EDM-S	C-EDM-M	C-EDM-L	C-EDM-XL	C-EDM-H
NSCNv2 [19]	10.87	[10.16](10.43)	[10.04](10.27)	[9.95](10.14)	[9.92](10.01)	[9.90](10.01)
DDPM [2]	3.17	[3.10](3.15)	[3.07](3.14)	[3.03](3.09)	[2.96](3.07)	[2.99](3.06)
SDE VE [3]	2.55	[2.51](2.52)	[2.48](2.50)	[2.46](2.47)	[2.43](2.46)	[2.41](2.46)
SDE VP [3]	2.20	[2.17](2.20)	[2.15](2.18)	[2.14](2.15)	[2.13](2.15)	[2.12](2.15)
EDM [20]	2.04	[1.96](1.99)	[1.93](1.95)	[1.90](1.94)	[1.87](1.93)	[1.87](1.93)

'Origin FID' refers to the results obtained from the target pre-trained DMs without correction. We optimize correction functions, C-EDM-$*$ (detailed in Table XI), parameterized by varying model sizes, with all the residual pairs constructed using EDM (the source pre-trained model). For example, C-EDM-XL denotes a correction function optimized on EDM, which is then used to enhance target pre-trained DMs in a plug-and-play manner. The values in [] are obtained using the correction function optimized based on the training mechanism proposed in this paper, and the values in () are presented in our conference paper [38]. Clearly, our correction functions, optimized using the newly proposed training mechanism—which includes the ODE sampler bank and score consistency loss—enable a notable transferable correction capability.

TABLE IV
EFFECTIVE RESIDUAL CORRECTION ON AFHQV2 AND CELEBA

AFHQv2 64 × 64		CelebA 64 × 64	
Pre-trained Model	FID↓	Pre-trained Model	FID↓
EDM VE [20]	2.16→[2.03](2.10)	DDPM++ [3]	2.32→[2.14](2.22)
EDM VP [20]	1.96→[1.87](1.91)	STDDPM [15]	1.90→[1.80](1.85)
FDM VP [81]	2.39→[2.22](2.30)	Soft Diffusion [82]	1.85→[1.76](1.80)
FDM VE [81]	47.30→[39.17](42.10)	INDM [83]	1.75→[1.69](1.71)

Models in the first column represents the baseline pre-trained DMs. The values before → represent the results from the baseline pre-trained DMs. The values after → indicate the results improved by our residual learning framework, where the values in [] are obtained using the correction function optimized based on the training mechanism proposed in this paper, and the values in () are presented in our conference paper [38].

TABLE V
EFFICIENT CORRECTION PERFORMANCE EVALUATED BY FID

Correction Function	Heavy Function	
Light Function↓	C-EDM-XL	C-EDM-H
C-EDM-T	2.03→[2.01](2.00)	2.03→[2.00](1.99)
C-EDM-S	1.99→[1.98](1.97)	1.99→[1.97](1.96)
C-EDM-M	1.95→[1.91](1.93)	1.95→[1.91](1.93)

We utilize the pre-trained heavy correction functions, formulated by C-EDM-XL and C-EDM-H, to train light correction functions that formulated by C-EDM-T, C-EDM-S and C-EDM-M respectively. The values before → represent the results obtained by using the correction functions in the first column to enhance the pre-trained EDM. The values after → represent the results distilled from the heavy correction functions, C-EDM-XL or C-EDM-H, where the values in [] are obtained from the correction function optimized based on the training mechanism proposed in this paper, and the values in () are presented in our conference paper [38].

TABLE VI
FID PERFORMANCE ON CIFAR-10 WHEN COMBINED WITH TRAINING-FREE FAST SAMPLERS

Method	NFE 12	NFE 24	NFE 48
DPM-Solver-2 (VP) [26]	5.28	3.02	2.69
DPM-Solver-2 (VP)*	5.12	2.94	2.65
DPM-Solver-2 (VP)**	5.04	2.91	2.62
DPM-Solver-3 (VP)	6.03	2.75	2.65
DPM-Solver-3 (VP)*	5.89	2.69	2.62
DPM-Solver-3 (VP)**	5.85	2.67	2.60
Method	**NFE 10**	**NFE 20**	**NFE 50**
DDIM [24]	13.36	6.84	4.67
DDIM**	12.77	6.56	4.33
DEIS (VP) [25]	4.17	2.86	2.57
DEIS (VP)*	4.11	2.82	2.55
DEIS (VP)**	4.08	2.80	2.54
DEIS (VE)	20.89	16.59	16.31
DEIS (VE)*	19.94	16.05	15.91
DEIS (VE)**	19.76	15.93	15.82

* indicates that the results have been improved by an optimized correct function trained using the method proposed in our conference paper [38], while ** represent the results enhanced by the correct function optimized in this paper. We verify that the our framework enables to improve arbitrary model sampling trajectory formulated by training-free fast samplers. The absence of * results for DDIM is because we did not conduct experiments on it in our conference paper.

work and verify its effectiveness under classifier-free guidance (CFG) sampling. To quantitatively compare the correction performance, we utilize standard metrics for evaluating DMs, including Fréchet Inception Distance (FID) [88] and Inception Score (IS) [89], and verify them on 50 K generated samples. The qualitative results are shown in Fig. 4.

Effective Residual Correction: For the evaluation of effective residual correction, we conduct experiments on several widely used benchmark datasets. In the experimental results on CIFAR-10, our framework enhances the generation performance, indicating that correction function effectively learns the residuals and, consequently, enhances the overall image quality, seen in Table II. Our framework also demonstrate the effectiveness on CelebA and AFHQv2 with FID significantly increased, shown in Table IV. Moreover, our framework demonstrates remarkable performance on high-resolution datasets, such as ImageNet 256 and 512, as detailed in Table I, where LDM [6] and DiT [5] are well-known latent diffusion frameworks. On the other hand, we also verify the experiments on correcting the sampling trajectory constructed by training-free fast samplers, results are displayed in Table VI. Overall, the proposed framework enhances various pre-trained DMs across different datasets, regardless of their model architecture.

Transferable Residual Correction: To demonstrate the transferable residual correction capability of the proposed framework, we select two datasets—CIFAR-10 and ImageNet 512—to conduct experiments. On CIFAR-10, we design five different versions of the correction function by allocating varying model channels to the EDM backbone. For example, we configure EDM-XL with 128 channels and EDM-L with 96 channels, as detailed in Table XI. Once the correction function is optimized, we can then employ one of them to enhance various DMs trained

TABLE VII
TRANSFERABLE RESIDUAL CORRECTION ON IMAGENET 512 × 512 WITH EDM2, EVALUATED USING FID

	Models Employed to Construct Residuals					
Pre-trained Model	C-EDM2-XS	C-EDM2-S	C-EDM2-M	C-EDM2-L	C-EDM2-XL	C-EDM2-XXL
EDM2-XS	3.53→[3.16]	3.53→[3.18]	3.53→[3.18]	3.53→[3.19]	3.53→[3.23]	3.53→[3.25]
EDM2-S	2.56→[2.46]	2.56→[2.34]	2.56→[2.37]	2.56→[2.40]	2.56→[2.38]	2.56→[2.41]
EDM2-M	2.25→[2.18]	2.25→[2.13]	2.25→[2.10]	2.25→[2.12]	2.25→[2.15]	2.25→[2.16]
EDM2-L	2.06→[2.03]	2.06→[2.00]	2.06→[2.01]	2.06→[1.96]	2.06→[1.98]	2.06→[2.01]
EDM2-XL	1.96→[1.94]	1.96→[1.94]	1.96→[1.94]	1.96→[1.92]	1.96→[1.90]	1.96→[1.91]
EDM2-XXL	1.91→[1.90]	1.91→[1.90]	1.91→[1.89]	1.91→[1.87]	1.91→[1.86]	1.91→[1.83]

EDM2 [17] refers to the improved version of EDM [20]. We design six versions of correction functions based on different EDM2 backbones. For instance, we use the EDM2-M backbone to parameterize the correction function C-EDM2-M, and then train C-EDM2-M using the pre-trained EDM2-M. We then apply the optimized C-EDM2-M to improve various pre-trained versions of EDM2, including EDM2-S, EDM2-M, EDM2-L, EDM2-XL, and EDM2-XXL, respectively. The values before → represent the results from the pre-trained DMs, while the values after → indicate the results improved by our framework.

TABLE VIII
COMPARISON WITH RECENT WORK [60] ON CELEBA AND FFHQ VIA FID EVALUATION ACROSS VARYING NFES

Method	CelebA 64 × 64					FFHQ 256 × 256		
	10	20	50	100	1000	100	200	500
DDIM	17.33	13.73	9.17	6.53	4.88	17.89	11.26	8.41
DDIM+CS [60]	7.80	5.11	2.23	2.11	1.98	11.89	7.31	4.02
DDIM+Ours	7.67	5.05	2.22	2.12	2.03	10.93	7.21	3.88
DDIM+Ours (optimized for DPM-Solver)	8.20	6.14	3.31	2.76	2.45	12.42	8.70	5.45
DPM-Solver	5.83	3.13	3.10	3.11	3.08	10.82	8.47	8.40
DPM-Solver+CS [60]	5.22	2.35	2.22	2.12	2.04	9.73	4.66	4.01
DPM-Solver+Ours	4.93	2.31	2.21	2.08	2.02	8.93	4.43	4.01
DPM-Solver+Ours (optimized for DDIM)	6.43	2.74	2.62	2.49	2.32	9.97	7.53	6.24

Clearly, our framework consistently outperforms [60] and, importantly, exhibits a transferable residual correction capability that is not available in [60]. To illustrate this transferability, we use the correction function optimized for DDIM to rectify DPM-Solver and, conversely, the correction function optimized for DPM-Solver to rectify DDIM. As shown above, our framework not only achieves superior residual correction performance compared to [60] but also demonstrates an impressive transferable residual correction capability.

TABLE IX
TRAINING COST COMPARISON ON CIFAR-10 AND IMAGENET

Metrics	GPU Hours ↓	Average NFEs ↓	Iterations	Batch Size
CIFAR-10				
EDM [20]	32 [90]	–	400K	512
C-EDM-H	25	8.1	100K	1024
C-EDM-XL	17	7.8	80K	1024
C-EDM-L	12	7.7	80K	1024
C-EDM-M	11	8.3	60K	1024
C-EDM-S	5	7.2	40K	1024
C-EDM-T	0.5	7.9	40K	1024
ImageNet 512×512				
DiT-XL/2-G [5]	1733	–	3000K	256
C-EDM2-XXL	950	11.7	100K	1024
C-EDM2-XL	800	13.2	80K	1024
C-EDM2-L	550	13.5	80K	1024
C-EDM2-M	400	12.9	70K	1024
C-EDM2-S	250	13.8	60K	1024
C-EDM2-XS	100	13.4	60K	1024

on CIFAR-10, with the detailed results shown in Table III. Analogously, since the EDM2 paper has already proposed six versions of EDM2s, we simply employ their backbones to serve as our correction functions. Specifically, we optimize them using our training framework based on the backbones of their corresponding pre-trained EDM2s and employ them to enhance the six versions of EDM2 trained on ImageNet 512. The significant improvements are illustrated in Table VII, which highlight the transferable correction performance of our framework.

Efficient Residual Correction: While our framework does not reduce the number of sampling steps, the inherently slow sampling process in DMs continues to impose a significant time cost for image generation. To address this issue to some extent, we propose an efficient residual correction approach that leverages a lightweight model to minimize correction latency. Specifically, we distill the correction capability of an optimized heavy correction function into a lightweight correction function, as demonstrated by the results in Table V. With only a slight reduction in performance, the lightweight correction functions are 4 to 64 times smaller than their heavier counterparts. Additionally, we utilize the lightweight correction function to enhance various DMs trained on CIFAR-10, showcasing excellent transferable correction performance, as presented in Table III. Clearly, our framework effectively achieves efficient residual correction.

Comparison with Recent Works: Since Lu et al. [60] also address cumulative errors in diffusion sampling, we conduct additional experiments on CelebA and FFHQ for comparison. Specifically, we optimize correction functions for DDIM and DPM-Solver and apply them to their respective sampling processes. As shown in Table VIII, our framework outperforms [60] across most NFE settings and additionally demonstrates transferable residual correction, a capability not available in [60]. Although the overall performance of transferable correction does not match that of specifically optimized correction functions, the results still demonstrate strong effectiveness and highlight the flexibility of our framework.

Boosting Guidance Mechanism: Since classifier-free guidance (CFG) is widely used in diffusion sampling for both controllability and quality, we also evaluate performance under this setting. To this end, we conduct experiments on ImageNet

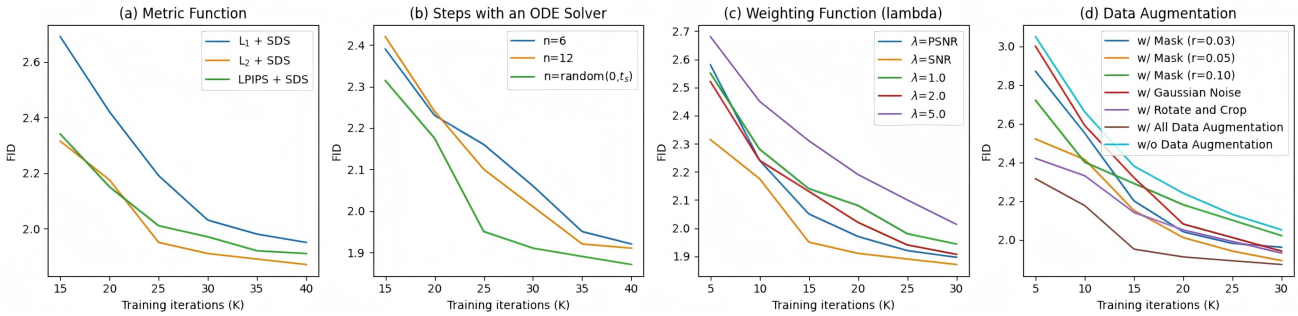


Fig. 5. Ablations of varying optimization strategies evaluated on CIFAR-10. (a) Correction functions optimized with different metric functions, where the L_2 norm combined with score consistency loss (SCS) achieves the best results. (b) The ODE solver is used to reverse x_t to x_i in n steps for simulating residuals, with the random selection of n yielding the best performance. (c) We design different weighting functions to optimize the model, including signal-to-noise ratio (SNR) [40], peak signal-to-noise ratio (PSNR) [91], and constant schedules. We empirically find that the SNR weighting schedule performs the best. (d) Combining all data augmentation methods, including mask pixel, adding Gaussian noise, rotation, and cropping, results in the best correction ability.

TABLE X
CFG SAMPLING RESULTS ON IMAGENET WITH DIT-XL/2-G

Method	FID ↓	sFID ↓
Class-Conditional ImageNet 256×256		
DiT-XL/2-G (cfg=1.25) [5]	3.22	5.28
DiT-XL/2-G (cfg=1.25) + Ours	3.13	5.33
DiT-XL/2-G (cfg=1.50) [5]	2.27	4.60
DiT-XL/2-G (cfg=1.50) + Ours	2.22	4.69
Class-Conditional ImageNet 512×512		
DiT-XL/2-G (cfg=1.25) [5]	4.64	5.77
DiT-XL/2-G (cfg=1.25) + Ours	4.55	5.83
DiT-XL/2-G (cfg=1.50) [5]	3.04	5.02
DiT-XL/2-G (cfg=1.50) + Ours	2.98	5.12

with DiT [5]. As shown in Table X, our framework consistently improves performance under CFG, demonstrating its strong modeling capability and practical utility.

C. Framework Cost

To comprehensively demonstrate the superiority and efficiency of our framework, in this section we primarily investigate the training and sampling overhead it incurs.

Residual Learning Cost: To systematically analyze our framework, we compare the actual training costs with those of the baselines—including GPU hours, NFEs used for residual construction, the number of training iterations, and batch size—as detailed in Table IX. Notably, EDM [20] does not clearly report GPU days, so we adopt the values provided in Fig. 4 of [90]. Moreover, we report the average NFEs over the entire training process, since the NFEs used to construct the residual vary across training iterations, being randomly determined within the time range $(\epsilon, t_S]$. Compared with the baselines, our correction functions incur lower training costs, especially when parameterized with a lightweight backbone. More importantly, our framework exhibits strong transferable residual correction capability, which further reduces training resource consumption, making it of substantial practical value.

Residual Correction Cost: To assess the sampling cost of our framework, we analyze the correction mechanism both intuitively and empirically. In practice, we set the correction threshold $c = \max(0.01, \epsilon)$ to determine when correction is applied, so it typically occurs before the end of the schedule. Under the EDM default schedule with Euler, generating one image requires 35 model evaluations. In our setup, correction is triggered when t approaches c, so we perform 32 sampling steps followed by one correction step—33 steps in total—rather than 35. For DDPM, our framework reduces the diffusion steps from 1000 to 980, plus one correction pass (981 total). Consequently, our framework reduces the overall sampling cost as the total sampling steps are decreased. To measure practical runtime, we used a single A100 GPU to generate 50 k images: the default schedule took 28 minutes, whereas our framework with the C-EDM-XL correction reduced this to 26.7 minutes. In summary, our framework naturally lowers the number of sampling steps and, correspondingly, the runtime.

D. Ablation Studies

To verify the effectiveness of different training settings, we conduct several ablations on CIFAR-10 using an untrained EDM backbone to parameterize the correction function.

Ablations on Training Settings: In the ablation studies on various optimization strategies, we perform four distinct experiments to validate their effectiveness. These experiments encompass the setting of optimization metric function, ODE reverse steps, data augmentation methods and weighting functions, detailed seen in Fig. 5. Additionally, we conduct ablation experiments on the enhancement of model convergence using newly introduced techniques. The empirical results presented in Table XII demonstrate the superiority of the proposed ODE sampler bank and score consistency loss.

Ablations on Model Parameters: For the ablations on model size, we design six different correction functions with different EDM backbone channels, seen in Table XI. We allocate varying training iterations and batch sizes to train these correction functions, depending on their different model sizes. It is reasonable that the best improvements are achieved by C-EDM-XL and C-EDM-H, given their larger model capacity.

Ablations on Threshold Settings: To elucidate the design principle behind choosing c, we conduct ablations on CIFAR-10

TABLE XI
ABLATION STUDY ON THE MODEL SIZE OF CORRECTION FUNCTIONS

Correction Function	Model Channels	Model Size	FID↓	IS↑	Training Iterations	Batch Size
C-EDM-H	256	204M	2.04→[1.87](1.93)	9.84→[9.99](9.91)	100K	1024
C-EDM-XL	128	51M	2.04→[1.87](1.93)	9.84→[9.99](9.89)	80K	1024
C-EDM-L	96	28.7M	2.04→[1.90](1.94)	9.84→[9.97](9.89)	80K	1024
C-EDM-M	64	12.7M	2.04→[1.93](1.95)	9.84→[9.89](9.86)	60K	1024
C-EDM-S	32	3.2M	2.04→[1.96](1.99)	9.84→[9.86](9.84)	40K	1024
C-EDM-T	16	0.8M	2.04→[1.99](2.03)	9.84→[9.84](9.80)	40K	1024

The experiments are conducted on CIFAR-10, with the residual construction employed using the pre-trained EDM [20]. All the correction functions are built on the EDM backbone, and the model size depends on the number of model channels. For instance, C-EDM-H is the correction function constructed by EDM backbone with 256 channels and the resulting model size is 204M. The values before → represent the results from the baseline pre-trained EDM. The values after → indicate the results improved by our residual learning framework, where the values in [] are obtained using the correction function optimized based on the training mechanism proposed in this paper, and the values in () are presented in our conference paper [38].

TABLE XII
ABLATION STUDY ON THE ADVANCED TRAINING APPROACH

Techniques	FID↓	IS↑
EDM [20]	2.04	9.84
C-EDM-L (Employed Correction Function)		
+Our Previous Conference Paper [38]	1.94(-0.10)	9.89(+0.05)
+ODE Sampler Bank	1.92(-0.12)	9.92(+0.08)
+Score Consistency Loss [20]	1.90(-0.14)	9.97(+0.13)
C-EDM-XL (Employed Correction Function)		
+Our Previous Conference Paper [38]	1.93(-0.11)	9.89(+0.05)
+ODE Sampler Bank	1.89(-0.15)	9.96(+0.12)
+Score Consistency Loss [20]	1.87(-0.17)	9.99(+0.15)

The results are obtained by conducting experiments on CIFAR-10 using EDM. We select two different correction functions to evaluate performance: C-EDM-L and C-EDM-XL. For each correction function, we present three different settings, including the results obtained in our conference paper [38], enhancements made using the ODE sampler bank, and the implementation of the score consistency loss.

TABLE XIII
ABLATION STUDY ON THRESHOLD c EVALUATED BY FID METRICS

Threshold c	0.13	0.075	0.043	0.024	0.013	0.007	0.002
FID	1.96	1.92	1.89	1.88	1.87	1.92	2.01

To analyze the sensitivity of threshold c, we conduct ablations on CIFAR-10 to rectify the pre-trained EDM [20]. Specifically, we set c to each of the last seven time values in the default EDM discretization schedule, $[0.13, 0.075, 0.043, 0.024, 0.013, 0.007, 0.002]$, and run experiments for each choice. Our correction remains effective when initiated with any of these values of c, improving upon the pre-trained EDM baseline (FID 2.04) and thereby demonstrating the robustness of our framework.

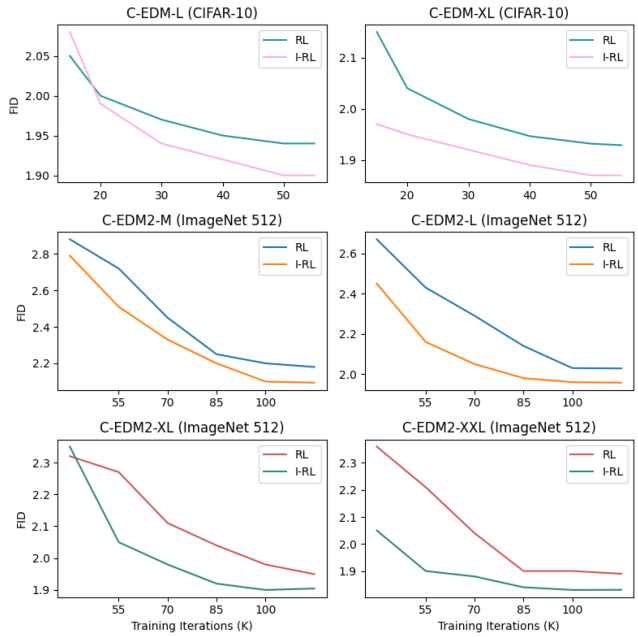


Fig. 6. Convergence comparisons between the framework in our previous paper and the newly proposed one. Here, 'RL' and 'I-RL' respectively represent the framework proposed in our previous paper [38] and its enhancement in this paper. For a thorough evaluation, we use C-EDM-L and C-EDM-XL for comparisons on CIFAR-10, while we utilize C-EDM2-M, C-EDM2-L, C-EDM2-XL, and C-EDM2-XXL to assess the enhancements on ImageNet 512. Clearly, the newly proposed optimization framework greatly boosts the convergence speed of the model parameters.

using the pre-trained EDM. As shown in Table XIII, our correction remains effective when initiated with any of the considered values of c, further demonstrating the robustness of our framework. Moreover, setting $c = 0.013$ yields the best results, which is consistent with our default rule $c = \max(0.01, \epsilon)$. Given its effectiveness, we therefore adopt the threshold rule in all experiments.

E. Comparison of Our Previous and Current Frameworks

To demonstrate the effectiveness of the proposed optimization framework, we carefully analyze the effects of the newly employed techniques. Specifically, we present the performance comparison in three aspects, which are convergence speed, correction performance, and transferability. The detailed convergence comparison is shown in Fig. 6. It is clear that the newly proposed method greatly improves convergence speed, requiring fewer training iterations. Moreover, both correction performance and transferability benefit from the ODE sampler bank and score consistency loss, demonstrating great potential for generative modeling. All these comparisons, including effective and transferable corrections, are shown in the tables, highlighting the discrepancies between the results in [] and ().

VIII. CONCLUSION

In this paper, we propose a novel residual learning framework based on a parameterized correction function designed to mitigate the residuals present in generated images. Importantly, the optimized correction function exhibits not only effective

residual correction performance but also remarkable transferable correction capabilities. To further improve our framework, we introduce an advanced training approach that incorporates the design of an ODE sampler bank to simulate varying magnitudes of residuals and a score consistency maintenance technique to facilitate model convergence. Following to this design philosophy, our improved residual learning framework achieves outstanding residual correction performance, as evidenced by extensive experimental results.

REFERENCES

[1] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 11918–11930.

[2] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 6840–6851.

[3] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–36.

[4] D. Podell et al., "SDXL: Improving latent diffusion models for high-resolution image synthesis," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–13.

[5] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 4195–4205.

[6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10684–10695.

[7] X. Wang et al., "VideoComposer: Compositional video synthesis with motion controllability," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, Art. no. 334.

[8] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, "DreamBooth: Fine tuning text-to-image diffusion models for subject-driven generation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 22500–22510.

[9] S. Liu, Z. Ren, S. Gupta, and S. Wang, "PhysGen: Rigid-body physics-grounded image-to-video generation," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 360–378.

[10] J. Lv et al., "GPT4Motion: Scripting physical motions in text-to-video generation via blender-oriented GPT planning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 1430–1440.

[11] J. Wynn and D. Turmukhambetov, "DiffusioNeRF: Regularizing neural radiance fields with denoising diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 4180–4189.

[12] S. Lee, J. Jo, and S. J. Hwang, "Exploring chemical space with score-based out-of-distribution generation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 18872–18892.

[13] D. Liu, Q. Li, A.-D. Dinh, T. Jiang, M. Shah, and C. Xu, "DiffAct++: Diffusion action segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 3, pp. 1644–1659, Mar. 2025.

[14] D. Kim, Y. Kim, S. J. Kwon, W. Kang, and I.-C. Moon, "Refining generative process with discriminator guidance in score-based diffusion models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 16567–16598.

[15] D. Kim, S. Shin, K. Song, W. Kang, and I.-C. Moon, "Soft truncation: A universal training technique of score-based diffusion model for high precision score estimation," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 11201–11228.

[16] C. Lu, K. Zheng, F. Bao, J. Chen, C. Li, and J. Zhu, "Maximum likelihood training for score-based diffusion ODEs by high order denoising score matching," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 14429–14460.

[17] T. Karras, M. Aittala, J. Lehtinen, J. Hellsten, T. Aila, and S. Laine, "Analyzing and improving the training dynamics of diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 24174–24184.

[18] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep unsupervised learning using nonequilibrium thermodynamics," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2256–2265.

[19] Y. Song and S. Ermon, "Improved techniques for training score-based generative models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 12438–12448.

[20] T. Karras, M. Aittala, T. Aila, and S. Laine, "Elucidating the design space of diffusion-based generative models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 26565–26577.

[21] B. Kim and J. C. Ye, "Denoising MCMC for accelerating diffusion-based generative models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 16955–16977.

[22] C. Luo, "Understanding diffusion models: A unified perspective," 2022, *arXiv:2208.11970*.

[23] A. Hyvärinen and P. Dayan, "Estimation of non-normalized statistical models by score matching," *J. Mach. Learn. Res.*, vol. 6, no. 4, pp. 695–709, 2005.

[24] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–22.

[25] Q. Zhang and Y. Chen, "Fast sampling of diffusion models with exponential integrator," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–33.

[26] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 5775–5787.

[27] T. Hang et al., "Efficient diffusion training via min-SNR weighting strategy," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 7441–7451.

[28] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 32211–32252.

[29] D. Kim et al., "Consistency trajectory models: Learning probability flow ODE trajectory of diffusion," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–33.

[30] T. Yin et al., "One-step diffusion with distribution matching distillation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 6613–6623.

[31] E. Hoogeboom, J. Heek, and T. Salimans, "Simple diffusion: End-to-end diffusion for high resolution images," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 13213–13232.

[32] C.-H. Chao et al., "Denoising likelihood score matching for conditional score-based data generation," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–24.

[33] D. Kingma and R. Gao, "Understanding diffusion objectives as the ELBO with simple data augmentation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 65484–65516.

[34] M. Hochbruck and A. Ostermann, "Exponential integrators," *Acta Numerica*, vol. 19, pp. 209–286, 2010.

[35] Y. Li and M. van der Schaar, "On error propagation of diffusion models," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–15.

[36] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "DreamFusion: Text-to-3D using 2D diffusion," in *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–20.

[37] Y. Lipman et al., "Flow matching guide and code," 2024, *arXiv:2412.06264*.

[38] J. Zhang, D. Liu, E. Park, S. Zhang, and C. Xu, "Residual learning in diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 7289–7299.

[39] P. Dhariwal and A. Nichol, "Diffusion models beat GANs on image synthesis," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 8780–8794.

[40] D. Kingma, T. Salimans, B. Poole, and J. Ho, "Variational diffusion models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 21696–21707.

[41] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013, *arXiv:1312.6114*.

[42] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–35.

[43] L. Zhang, A. Rao, and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3836–3847.

[44] R. Du, D. Chang, T. Hospedales, Y.-Z. Song, and Z. Ma, "DemoFusion: Democratising high-resolution image generation with no $$$," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 6159–6168.

[45] L. Zhang, A. Rao, and M. Agrawala, "Scaling in-the-wild training for diffusion-based illumination harmonization and editing by imposing consistent light transport," in *Proc. Int. Conf. Learn. Representations*, 2025, pp. 1–18.

[46] G. Sun, W. Liang, J. Dong, J. Li, Z. Ding, and Y. Cong, "Create your world: Lifelong text-to-image diffusion," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 9, pp. 6454–6470, Sep. 2024.

[47] D. Epstein, A. Jabri, B. Poole, A. Efros, and A. Holynski, "Diffusion self-guidance for controllable image generation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 16222–16239.

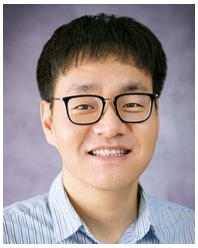
[48] N. Tumanyan, M. Geyer, S. Bagon, and T. Dekel, "Plug-and-play diffusion features for text-driven image-to-image translation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 1921–1930.

[49] H. Chen et al., "VideoCrafter2: Overcoming data limitations for high-quality video diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 7310–7320.

[50] L. Khachatryan et al., "Text2Video-Zero: Text-to-image diffusion models are zero-shot video generators," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 15954–15964.
[51] B. Cai et al., "3D scene creation and rendering via rough meshes: A lighting transfer avenue," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 9, pp. 6292–6305, Sep. 2024.
[52] C.-H. Lin et al., "Magic3D: High-resolution text-to-3D content creation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 300–309.
[53] M. Zhang et al., "MotionDiffuse: Text-driven human motion generation with diffusion model," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 4115–4128, Jun. 2024.
[54] M. Li et al., "Latent diffusion enhanced rectangle transformer for hyperspectral image restoration," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 1, pp. 549–564, Jan. 2025.
[55] C. Cao, H. Zhang, Y. Lu, P. Wang, and Y. Zhang, "Scene-dependent prediction in latent space for video anomaly detection and anticipation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 1, pp. 224–239, Jan. 2025.
[56] A. Lou, C. Meng, and S. Ermon, "Discrete diffusion language modeling by estimating the ratios of the data distribution," in *Proc. Int. Conf. Mach. Learn.*, 2024, Art. no. 1333.
[57] Y. Luo, Q. Yang, Y. Fan, H. Qi, and M. Xia, "Measurement guidance in diffusion models: Insight from medical image synthesis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 7983–7997, Dec. 2024.
[58] Z. Zhou, D. Chen, C. Wang, and C. Chen, "Fast ODE-based sampling for diffusion models in around 5 steps," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 7777–7786.
[59] T. Wang, Z. Dou, C. Bao, and Z. Shi, "Diffusion mechanism in residual neural network: Theory and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 2, pp. 667–680, Feb. 2024.
[60] H. Lu, A. A. Salah, and R. Poppe, "Compensation sampling for improved convergence in diffusion models," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 183–201.
[61] H. Wang, J. Zhang, H. Guo, D. Wang, J. Ma, and B. Du, "DGSolver: Diffusion generalist solver with universal posterior sampling for image restoration," 2025, *arXiv:2504.21487*.
[62] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," in *Proc. Int. Conf. Learn. Representations*, 2022, pp. 1–21.
[63] C. Meng et al., "On distillation of guided diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 14297–14306.
[64] M. Caron et al., "Emerging properties in self-supervised vision transformers," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 9650–9660.
[65] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
[66] A. R. Zamir, A. Sax, W. Shen, L. J. Guibas, J. Malik, and S. Savarese, "Taskonomy: Disentangling task transfer learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 3712–3722.
[67] N. Kumari, R. Zhang, E. Shechtman, and J.-Y. Zhu, "Ensembling off-the-shelf models for GAN training," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10651–10662.
[68] Z. Gu, W. Li, J. Huo, L. Wang, and Y. Gao, "LofGAN: Fusing local representations for few-shot image generation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 8463–8471.
[69] S. Kornblith, J. Shlens, and Q. V. Le, "Do better imagenet models transfer better?," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2661–2671.
[70] U. Ojha et al., "Few-shot image generation via cross-domain correspondence," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 10743–10752.
[71] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1180–1189.
[72] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, "Adversarial discriminative domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7167–7176.
[73] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–15.
[74] E. Xie et al., "DiffFit: Unlocking transferability of large diffusion models via simple parameter-efficient fine-tuning," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 4230–4239.
[75] P. Vincent, "A connection between score matching and denoising autoencoders," *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, 2011.
[76] Y. Kim, G. Hwang, J. Zhang, and E. Park, "DiffuseHigh: Training-free progressive high-resolution image synthesis through structure guidance," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 4338–4346.
[77] G. Kim, T. Kwon, and J. C. Ye, "DiffusionClip: Text-guided diffusion models for robust image manipulation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 2426–2435.
[78] Z.-G. Liu, Y.-M. Fu, Q. Pan, and Z.-W. Zhang, "Orientational distribution learning with hierarchical spatial attention for open set recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 7, pp. 8757–8772, Jul. 2023.
[79] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 586–595.
[80] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 16000–16009.
[81] Z. Wu, P. Zhou, K. Kawaguchi, and H. Zhang, "Fast diffusion model," 2023, *arXiv:2306.06991*.
[82] G. Daras, M. Delbracio, H. Talebi, A. G. Dimakis, and P. Milanfar, "Soft diffusion: Score matching for general corruptions," 2022, *arXiv:2209.05442*.
[83] D. Kim, B. Na, S. J. Kwon, D. Lee, W. Kang, and I.-C. Moon, "Maximum likelihood training of implicit nonlinear diffusion model," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 32270–32284.
[84] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Master's thesis, Univ. Tront, pp. 1–60, 2009.
[85] Z. Liu, P. Luo, X. Wang, and X. Tang, "Deep learning face attributes in the wild," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3730–3738.
[86] Y. Choi, Y. Uh, J. Yoo, and J.-W. Ha, "StarGAN v2: Diverse image synthesis for multiple domains," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8188–8197.
[87] A. Van Den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1747–1756.
[88] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local nash equilibrium," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6629–6640.
[89] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 2234–2242.
[90] W. Harvey and F. Wood, "Visual chain-of-thought diffusion models," 2023, *arXiv:2303.16187*.
[91] J. Choi, J. Lee, C. Shin, S. Kim, H. Kim, and S. Yoon, "Perception prioritized training of diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 11472–11481.

Junyu Zhang received the BS, MS, and PhD degrees from Central South University, China. He is currently working toward the second PhD degree with the Department of Electrical and Electronic Engineering, Sungkyunkwan University, South Korea. His research interests include generative artificial intelligence, particularly diffusion models, and flow matching, as well as downstream tasks such as long-video generation and 3D scene synthesis.

Daochang Liu received the BE degree from Tongji University, in 2017, and the PhD degree from Peking University, in 2022. He is a lecturer with the School of Physics, Mathematics and Computing, University of Western Australia. He was a postdoctoral researcher with the University of Sydney. His research interests include rooted in understanding human action and skills with computer vision, which also involves fundamental problems in generative models and applications in healthcare.



Eunbyung Park received the BS degree in computer science from Kyung Hee University, in 2009, the MS degree in computer science from Seoul National University, in 2011, and the PhD degree in computer science from the University of North Carolina at Chapel Hill, in 2019. He is currently an assistant professor with the Department of Artificial Intelligence, Yonsei University, South Korea. Before joining Yonsei University, he was an assistant professor with the Department of Electrical and Electronic Engineering, Sungkyunkwan University, a research scientist with Nuro, and an applied scientist with Microsoft. He served/is serving as an area chair of NeurIPS, CVPR, and ICLR.

Shichao Zhang (Senior Member, IEEE) received the PhD degree from Deakin University, Australia. He is a China National distinguished professor with the Central South University, China. His research interests include information quality and pattern discovery. He has published 100 international journal papers and more than 80 international conference papers. He is a CI for 18 competitive national grants. He served/is serving as an associate editor of the *ACM Transactions on Knowledge Discovery From Data*, *IEEE Transactions on Knowledge and Data Engineering*, *Knowledge and Information Systems*.

Chang Xu (Senior Member, IEEE) received the PhD degree from Peking University, China. He is ARC future fellow and associate professor with the School of Computer Science, University of Sydney. He received the University of Sydney Vice-Chancellor's Award for Outstanding Early Career Research. His research interests lie in machine learning algorithms and applications in computer vision. He has published more than 100 papers in prestigious journals and top tier conferences. He has received several paper awards, including Distinguished Paper Award in AAAI 2023 and Distinguished Paper Award in IJCAI 2018. He served as an area chair of NeurIPS, ICML, ICLR, KDD, CVPR, and MM, as well as a senior PC member of AAAI and IJCAI. In addition, he served as an associate editor of *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Multimedia*, and *Transactions on Machine Learning Research*. He has been named a Top Ten distinguished senior PC member in IJCAI 2017 and an Outstanding associate editor of the *IEEE Transactions on Multimedia* in 2022.